are provably minimal, in the sense that the set of explored processes from some scheduling point must be a source set in order to guarantee exploration of all Mazurkiewicz traces. When a minimal persistent set contains more elements than the corresponding source set, the additional elements will always initiate sleep set blocked explorations. This implies that a necessary and sufficient condition for the correctness of *any* DPOR algorithm is that the set of explored process steps is a source set. For instance, correctness of persistent-set-based DPOR algorithms is often proved by establishing that the set of explored process steps is always a persistent set. The results of this article imply that in such proofs, it is enough to show the weaker property that this set is always a source set. We thus claim that source sets are the "right" conceptual foundation for developing DPOR techniques.

To show the power of source sets, we develop a simple DPOR algorithm, called *source-DPOR*, which is based on source sets. It is derived by modifying the classical persistent-set-based DPOR algorithm by Flanagan and Godefroid [2005b] so that persistent sets are replaced by source sets. The modification only consists in a small change to a single test in the algorithm. The power of source sets can be observed by noting that with this small modification, *source-DPOR* achieves significantly better reduction in the number of explored interleavings than the classical DPOR algorithm. In fact, *source-DPOR* explores the minimal number of interleavings for a large number of our benchmarks in Section 11.

To further demonstrate the power of source sets, we use them to develop a provably optimal DPOR algorithm. This is done by combining source sets with a novel mechanism, called *wakeup trees*, thereby deriving the algorithm *optimal-DPOR*. Wakeup trees control the initial steps of future explorations, implying that *optimal-DPOR* never encounters any sleep set blocked (i.e., redundant) exploration. An important feature of wakeup trees is that they are simple data structures that are constructed from already explored interleavings, hence they do not increase the amount of exploration. On the other hand, they allow to reduce the number of explored executions. In our benchmarks, maintenance of the wakeup trees reduces total exploration time when *source-DPOR* encounters sleep set blocked explorations and furthermore it never requires more than 10% of additional time in the cases where there are none or only a few sleep set blocked explorations. Memory consumption is practically always the same between our two DPOR algorithms, and in our experience the space cost of maintaining wakeup trees is very small. Still, as we will show, one can construct programs where the size of wakeup trees grows exponentially, and consequently the memory requirements of optimal-DPOR can be considerably worse than those of the source-DPOR algorithm. However, each branch in the wakeup tree is a prefix of some actually explored execution, and hence the size of the wakeup trees can never be larger than the size of all explored executions. Therefore, memory consumption is a problem only in situations where any DPOR-based algorithm needs to explore an exponential set of traces and does not seem to affect the time performance of the optimal algorithm; see Section 9.

We show the applicability of our algorithms to a wide range of computation models, including shared variables and message passing, by formulating them in a general setting, which only assumes that we can compute a *happens-before* relation (also called a causal ordering) between the events in an execution. For systems with shared variables, the happens-before relation can be based on the variables that are accessed or modified by events. For message passing systems, the happens-before relation can be based on correlating the transmission of a message with the corresponding reception. Our approach allows to make finer distinctions, leading to better reduction, than many other approaches that define a happens-before relation which is based on program statements, possibly taking into account the local state in which they are executed [Clarke et al. 1999; Flanagan and Godefroid 2005b; Godefroid 1996, 2005; Lauterburg et al. 2010; Tasharofi et al. 2012; Valmari